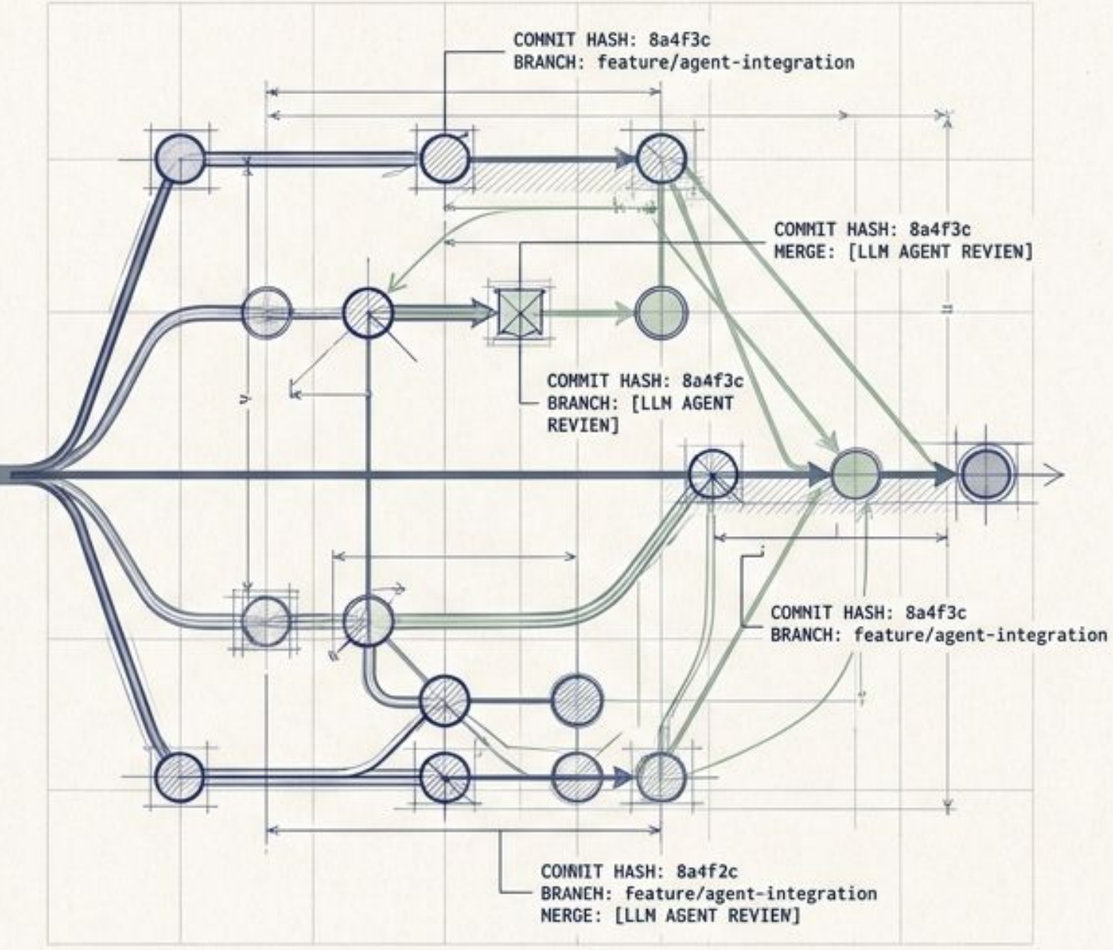
The End of Trunk-Based Development

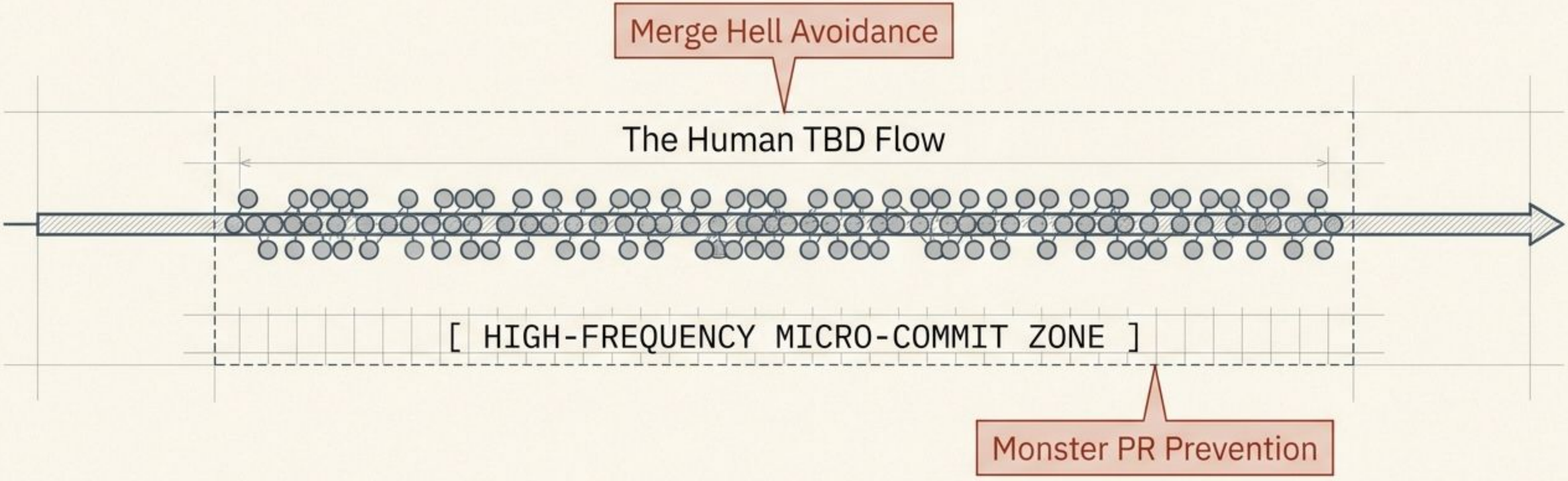
Why LLM agents will force engineering teams to abandon our most sacred workflow—and return to Gitflow.



NotebookLM

We adopted a defensive stance to survive our own cognitive limits.

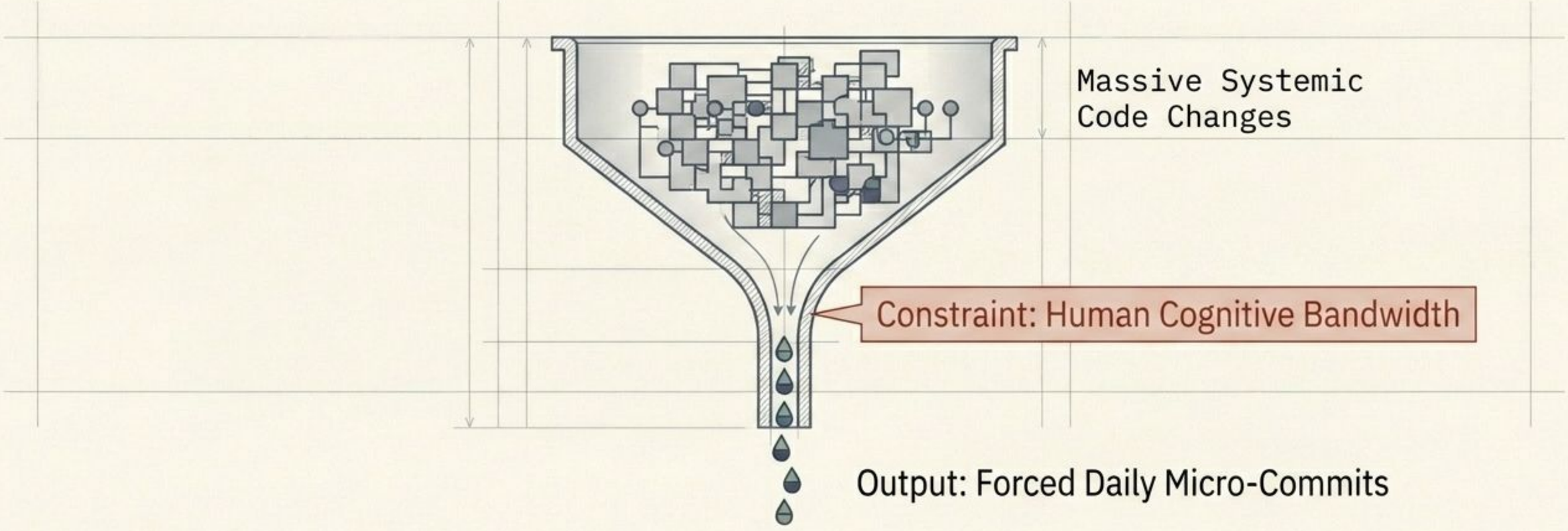
Trunk-Based Development has been our gold standard. The logic was undeniable: to avoid merge hell and the impossibility of reviewing long-lived branches, we forced ourselves to merge in small, daily increments. It was a necessary survival mechanism.



NotebookLM

Trunk-Based Development is an optimisation for human limitations.

We hate long-lived branches because humans read code line-by-line, sequentially. We cannot hold massive, concurrent systemic changes in working memory. The constraints that made TBD necessary were biological, not architectural.

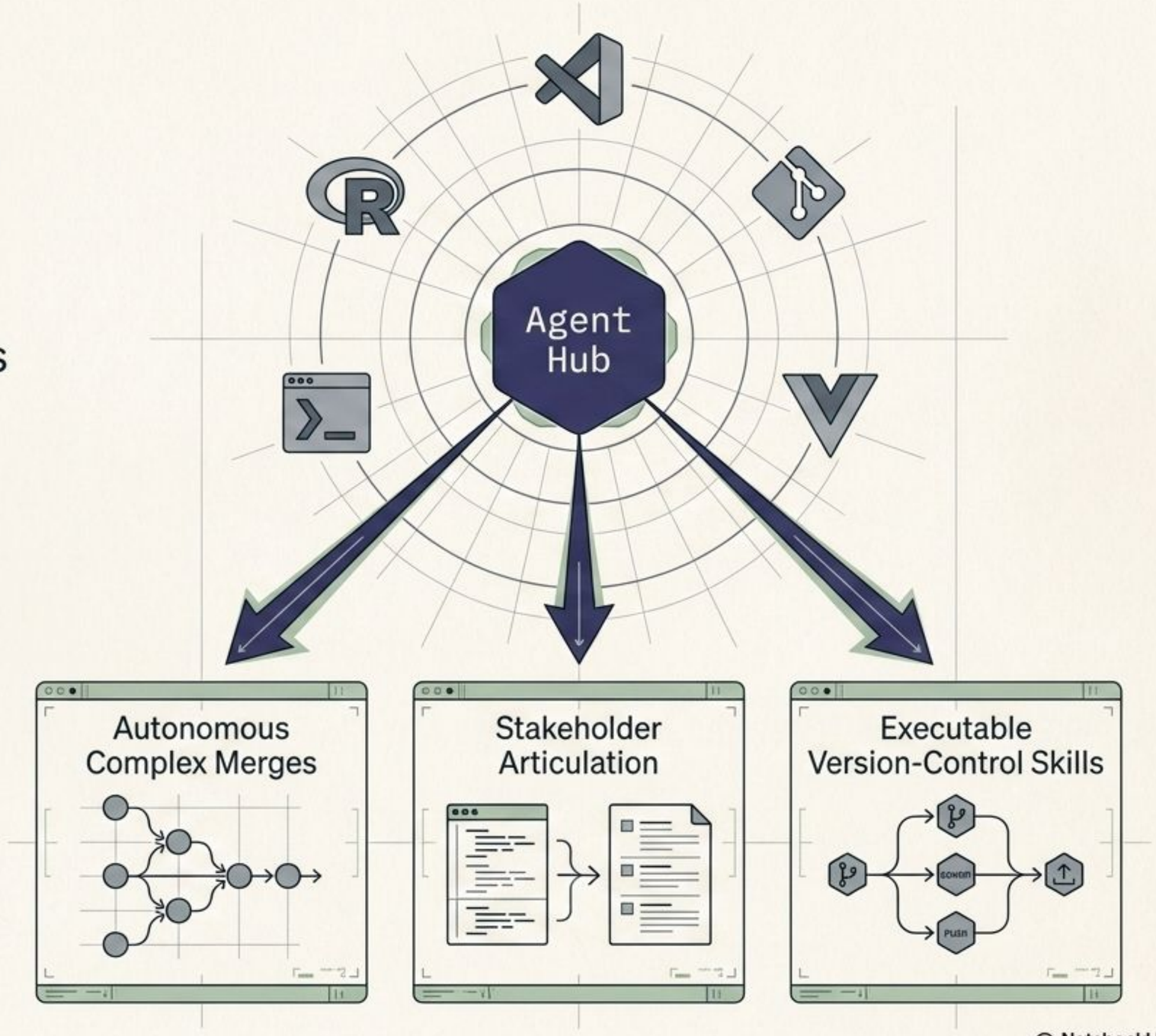


NotebookLM

The constraints that demanded TBD have vanished.

LLM-enabled IDEs and asynchronous agents process information differently. They autonomously handle massive merges, articulate vast code changes, and execute complex operations natively.

Isolated flow-based workflows are now superior.



NotebookLM

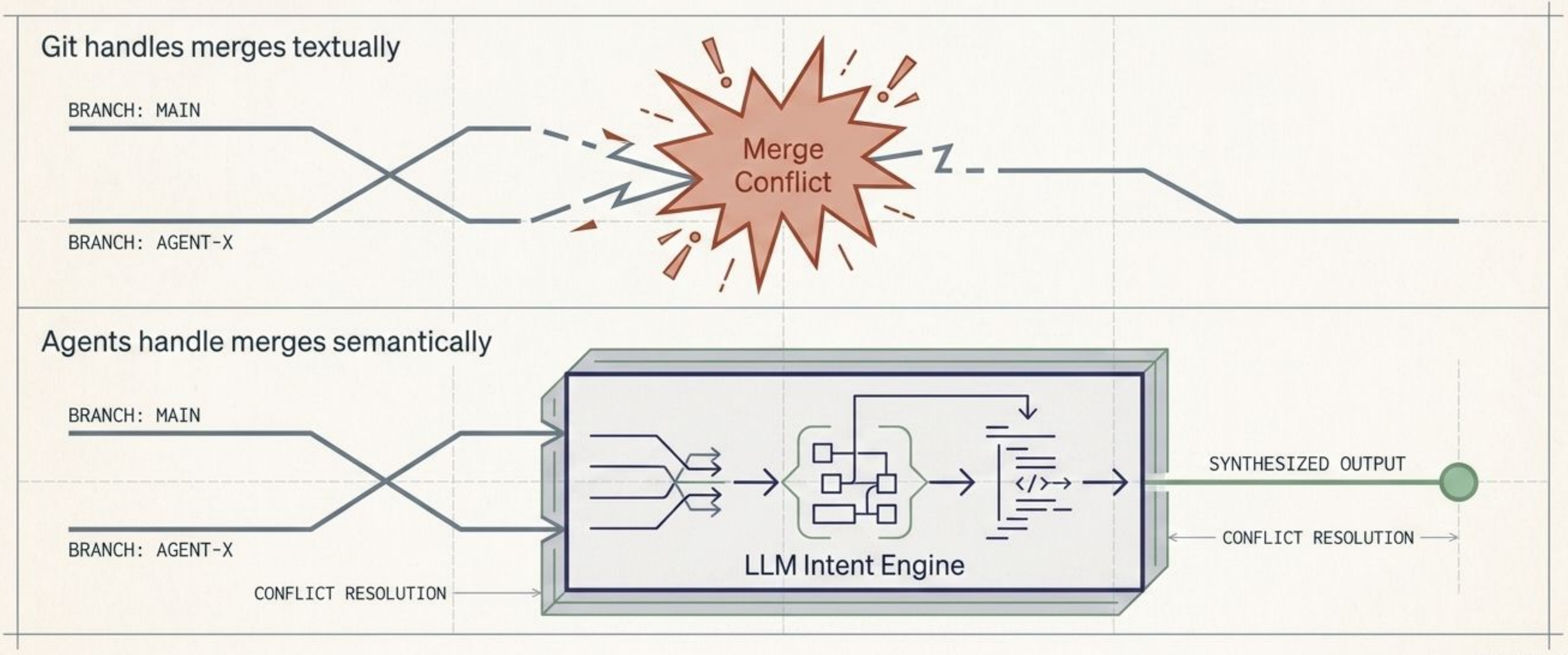
Why agent capabilities unlock the return to Gitflow.

Challenge	Human Limits (The Past)	Agent Unlock (The Future)
Concurrent Work	Breaks trunk, high collision	Operates safely in isolated sandboxes
Code Drift	Textual, line-by-line conflict	Semantic, intent-based resolution
PR Reviews	Cognitive overload from raw diffs	Generates visual, multi-media artifacts
Workflow Overhead	Manual CLI operations	Automated via executable skills
	Outcome: Requires TBD	Outcome: Unlocks Gitflow

NotebookLM

The end of Merge Hell: Semantic over Textual Merging.

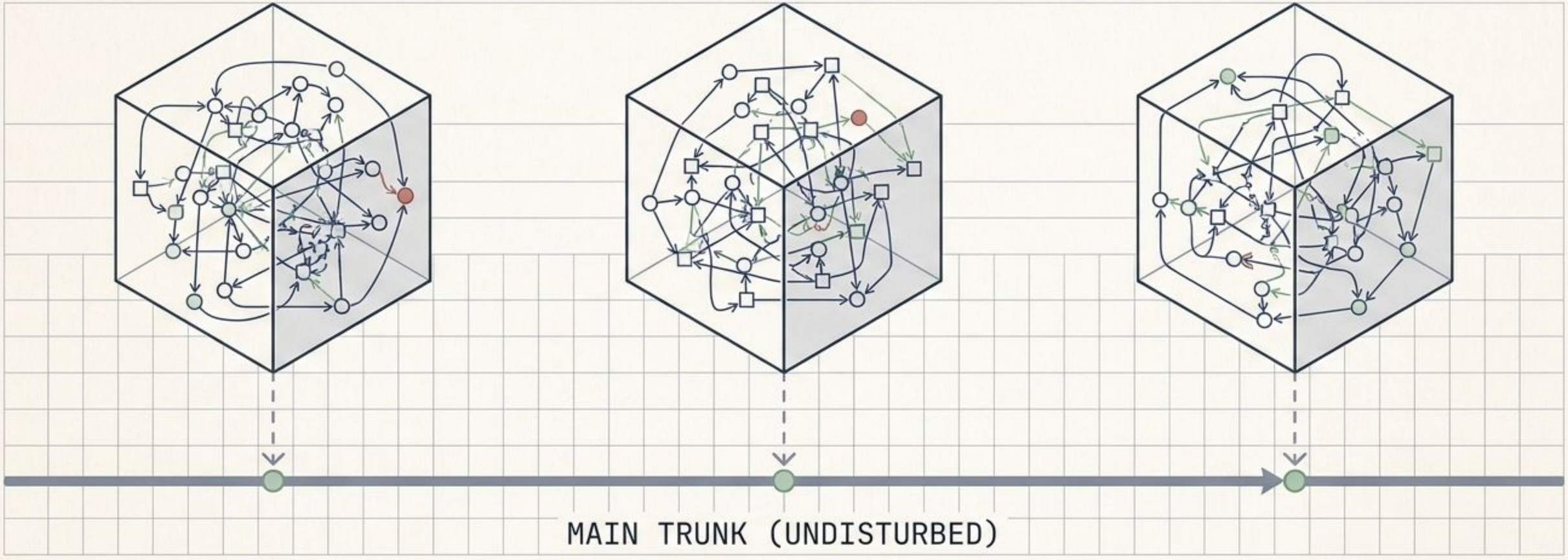
If a long-lived agent branch drifts from main over two weeks, the agent doesn't just read the conflicting text. It understands the **architectural goals** of both branches and synthetically rewrites the code to resolve the conflict.



NotebookLM

Asynchronous agents require sandboxes, not the trunk.

Deploying multiple asynchronous agents directly to the trunk creates a constantly breaking build. Forking and feature branching provide perfect silos. An agent can plan, code, test, and safely break things autonomously until flawless.



NotebookLM

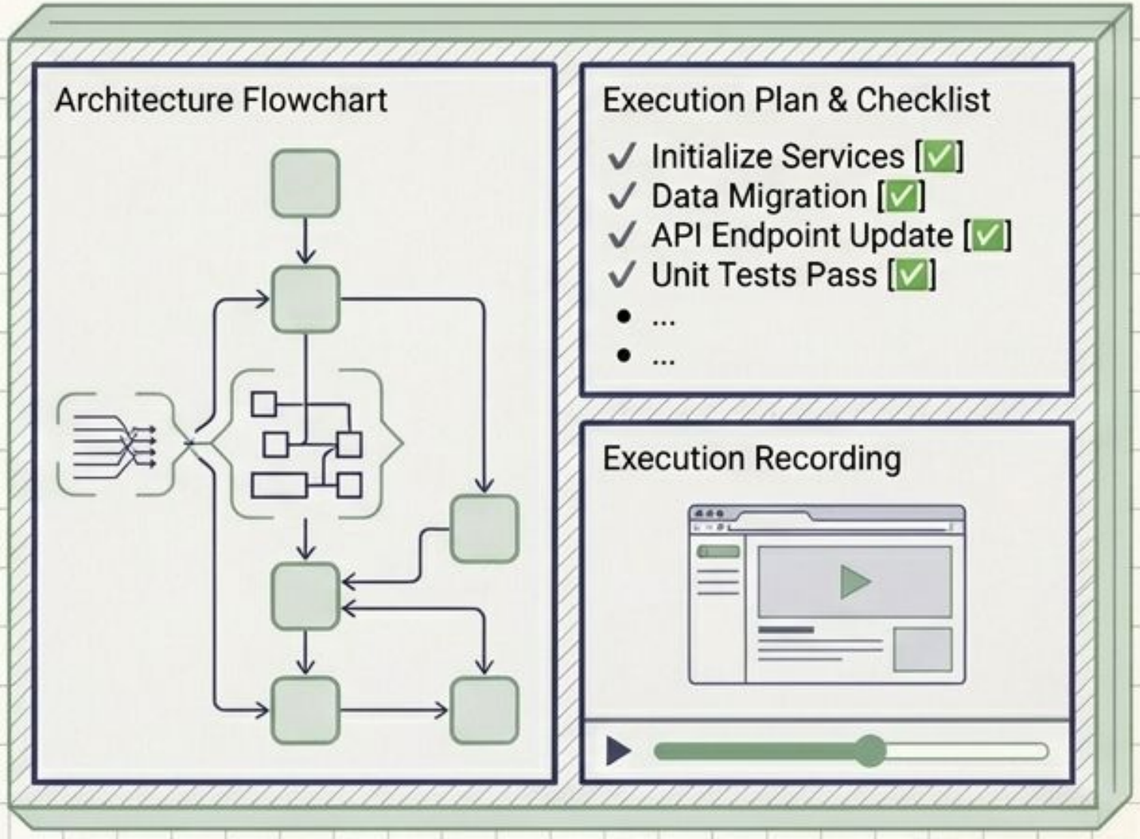
Rich artifacts eliminate the Monster PR bottleneck.

When an agent finishes a 3-week branch, we no longer review every line of code. We review comprehensive artifacts, watch the execution, and let the agent incorporate feedback asynchronously.

The Monster PR

Raw 5,000-line code diff

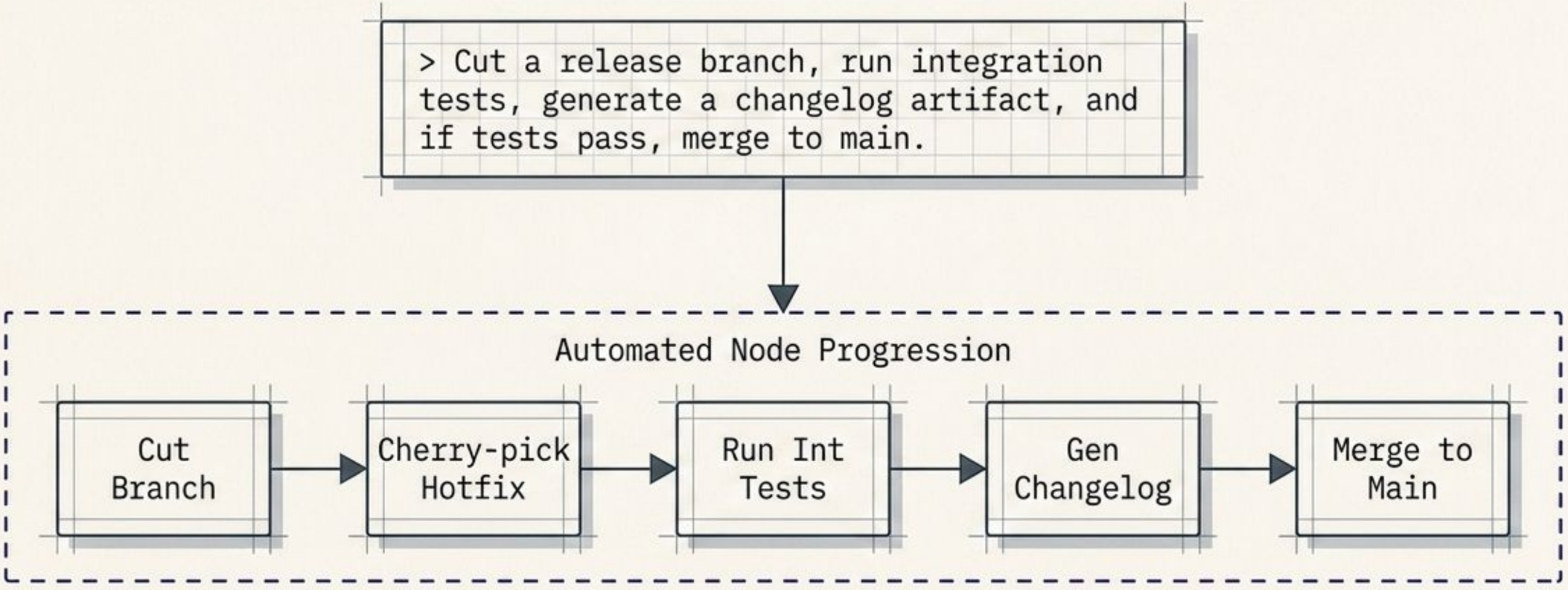
The Artifact UI



NotebookLM

Gitflow overhead becomes an executable agent skill

Complex operations used to require significant manual overhead. We can now embed these as permanent Skills, triggered by natural language.

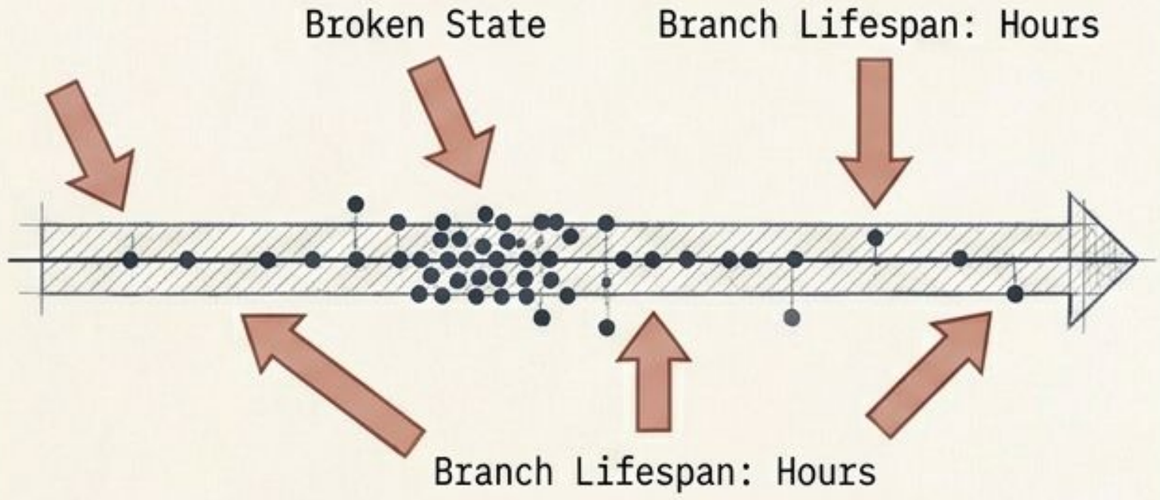


NotebookLM

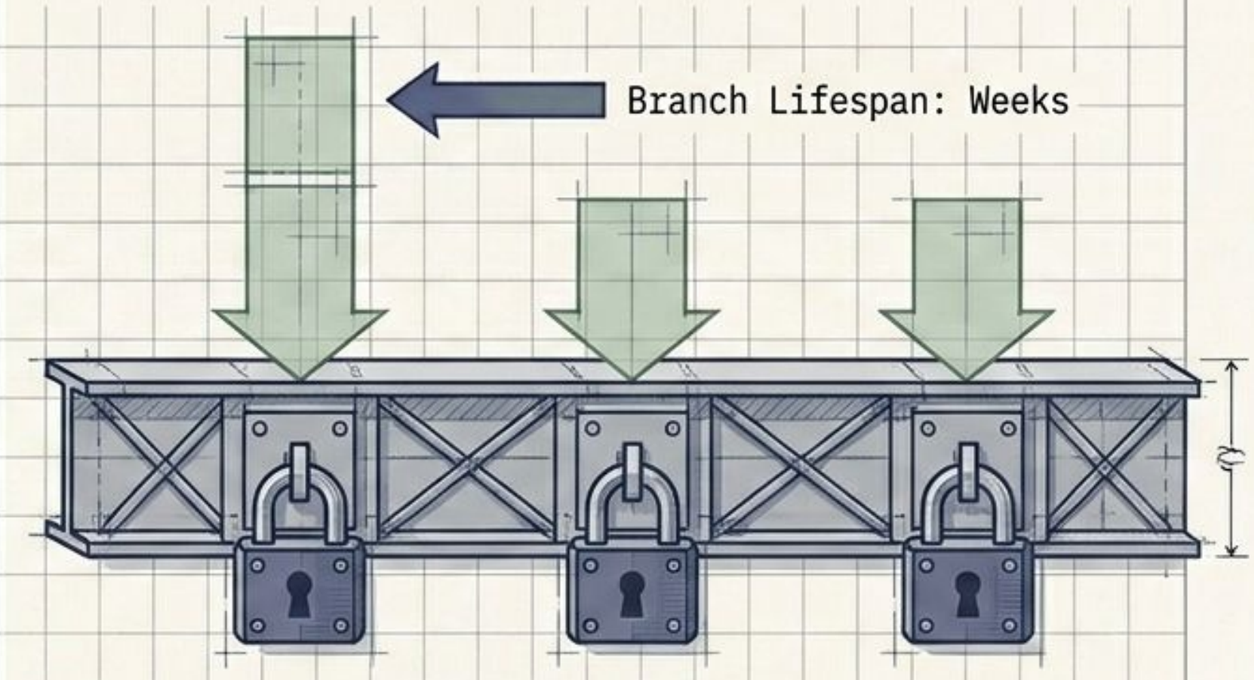
The trunk reverts to a pristine vault.

In an agent-driven world, the main branch is no longer a fragile lane for human work-in-progress. It is a locked, pristine vault strictly reserved for perfectly verified, completed code.

Old Model: Work-In-Progress Lane



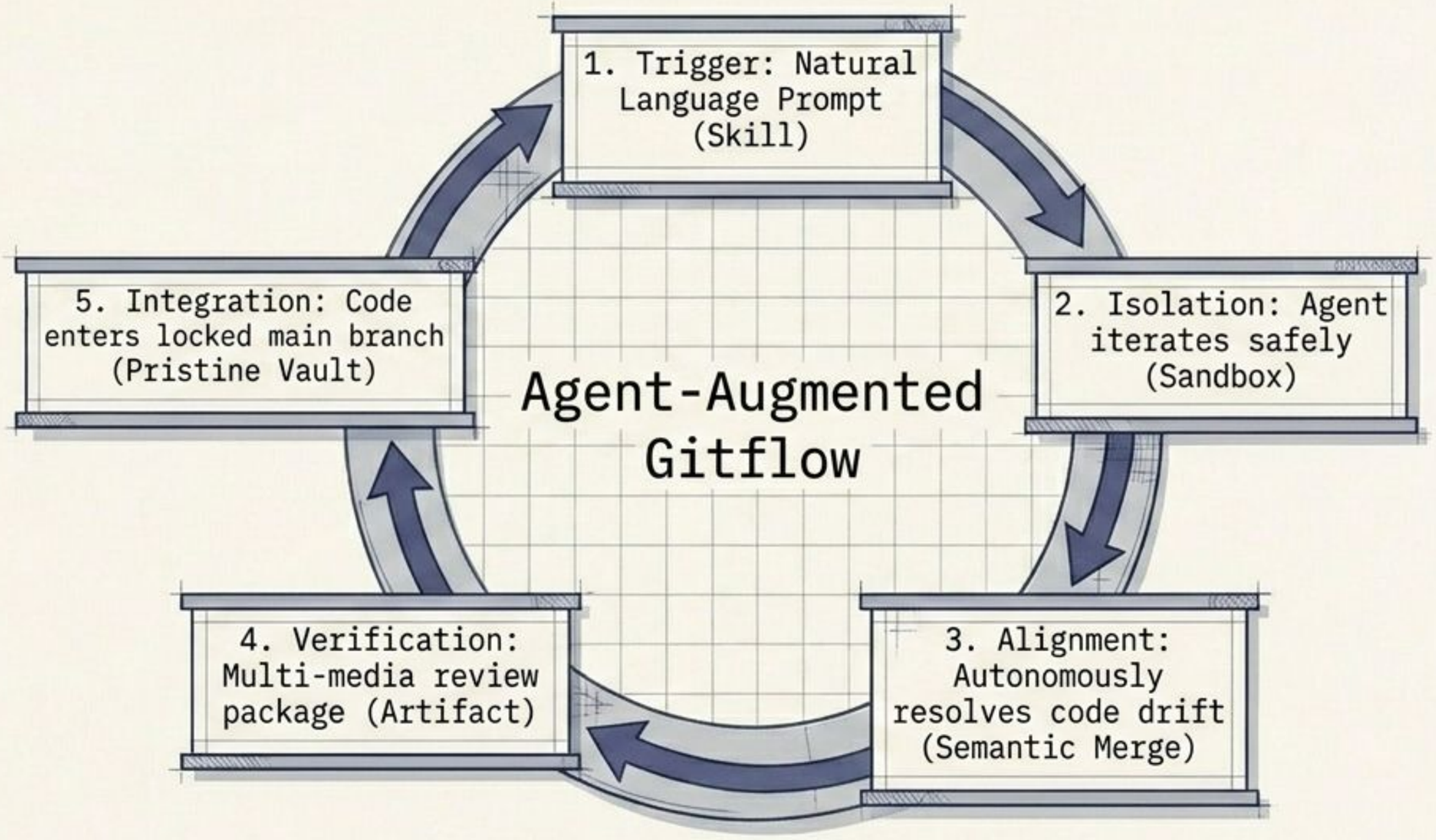
New Model: The Pristine Vault



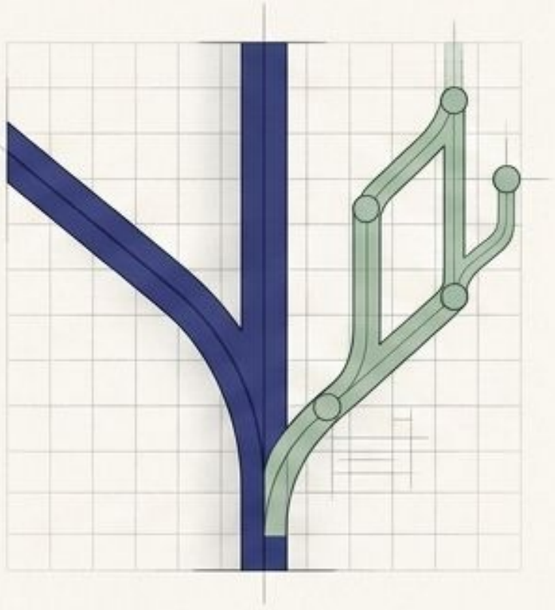
NotebookLM

The closed-loop system of Agent-Augmented Gitflow.

These four capabilities do not operate in isolation. Together, they create a cyclical engine that replaces human **cognitive strain** with asynchronous, autonomous heavy lifting.



NotebookLM



Stop constraining agents to human workflows.

Trunk-Based Development was an elegant solution to human limitations.
But as we shift to agent-driven development, the messy, long-term construction work must be offloaded to agents operating in deep, isolated branches.
The constraints are gone. It is time to evolve the flow.

NotebookLM